

# MATLAB Angles

This worksheet contains a list of MATLAB commands that you may want/need for the problem at the end. Some of these commands will be familiar, but they are included to contrast them with new ones. Enter each of these, and pay careful attention to the outputs to see exactly what the commands are doing.

For each of the following, let  $\mathbf{A}$  be the matrix  $\begin{pmatrix} 1 & 3 & -6 & 1 \\ -1 & 1 & 5 & -6 \\ -2 & 2 & -2 & 3 \\ 4 & 0 & 5 & 2 \end{pmatrix}$  and  $\mathbf{B}$  be the vector  $\begin{pmatrix} 1 \\ 5 \\ 3 \\ 2 \end{pmatrix}$ .

1 `A*A`

Matlab

## Effect 1

Computes the matrix product  $\mathbf{A}\mathbf{A}$ .

$$\mathbf{A}\mathbf{A} = \begin{pmatrix} 14 & -6 & 26 & -33 \\ -36 & 8 & -29 & -4 \\ 12 & -8 & 41 & -14 \\ 2 & 22 & -24 & 23 \end{pmatrix}$$

1 `A.*A`

Matlab

## Effect 2

Computes the products of corresponding entries between  $\mathbf{A}$  and  $\mathbf{A}$ , essentially squaring every entry in  $\mathbf{A}$ :

$$\begin{pmatrix} 1 & 9 & 36 & 1 \\ 1 & 1 & 25 & 36 \\ 4 & 4 & 4 & 9 \\ 16 & 0 & 25 & 4 \end{pmatrix}$$

# MATLAB Angles

1 `sum(A)`

Matlab

## Effect 3

Computes the column sums of each column in **A** as a row vector:

$$(2 \ 6 \ 2 \ 0)$$

Combining two of the above, what does `sum(A.*A)` do, and why would the output be important, based on what we've previously done in class?

## Answer 4

The code snippet `sum(A.*A)` computes the column sums of each column in **A** after all of **A**'s entries are squared:

$$(22 \ 14 \ 90 \ 50)$$

This output is important because each entry is a dot product of a column vector of **A** with itself. Essentially:

$$\begin{aligned} (22 \ 14 \ 90 \ 50) &= (\mathbf{a}_1 \cdot \mathbf{a}_1 \quad \mathbf{a}_2 \cdot \mathbf{a}_2 \quad \mathbf{a}_3 \cdot \mathbf{a}_3 \quad \mathbf{a}_4 \cdot \mathbf{a}_4) \\ &= (\|\mathbf{a}_1\|^2 \quad \|\mathbf{a}_2\|^2 \quad \|\mathbf{a}_3\|^2 \quad \|\mathbf{a}_4\|^2) \end{aligned}$$

1 `[A B]`

Matlab

## Effect 5

Creates a matrix using the columns of **A**, then augments the result with **B**.

$$\left( \begin{array}{cccc|c} 1 & 3 & -6 & 1 & 1 \\ -1 & 1 & 5 & -6 & 5 \\ -2 & 2 & -2 & 3 & 3 \\ 4 & 0 & 5 & 2 & 2 \end{array} \right)$$

# MATLAB Angles

1 `vertcat(A,B')`

Matlab

## Effect 6

Creates a matrix using the rows of  $\mathbf{A}$ , then concatenates the row vector  $\mathbf{B}^\top$  to the bottom.

$$\begin{pmatrix} 1 & 3 & -6 & 1 \\ -1 & 1 & 5 & -6 \\ -2 & 2 & -2 & 3 \\ 4 & 0 & 5 & 2 \\ 1 & 5 & 3 & 2 \end{pmatrix}$$

1 `R = rand(4,6)`

Matlab

## Effect 7

Creates a  $4 \times 6$  matrix  $\mathbf{R}$  where each entry is given by  $0 \leq R_{ij} < 1$ .

1 `sortrows(A)`

Matlab

## Effect 8

Sorts the rows of  $\mathbf{A}$  by their first entry in each row in ascending order.

$$\begin{pmatrix} -2 & 2 & -2 & 3 \\ -1 & 1 & 5 & -6 \\ 1 & 3 & -6 & 1 \\ 4 & 0 & 5 & 2 \end{pmatrix}$$

# MATLAB Angles

What do these variations on the `sortrows` command do? You may want to test your conjectures by trying these commands on different matrices.

```
1 [E, index] = sortrows(A)
```

Matlab

## Effect 9

Sorts the rows of **A** by their first entry in ascending order and initializes **E** to that result. Then, column vector **index** is initialized such that the value of each *i*th entry is the original row index of the corresponding *i*th row in **E**.

$$\mathbf{E} = \begin{pmatrix} -2 & 2 & -2 & 3 \\ -1 & 1 & 5 & -6 \\ 1 & 3 & -6 & 1 \\ 4 & 0 & 5 & 2 \end{pmatrix} \quad \text{index} = \begin{pmatrix} 3 \\ 2 \\ 1 \\ 4 \end{pmatrix}$$

```
1 sortrows(A, "descend")
```

Matlab

## Effect 10

Sorts the rows of **A** by their first entry in descending order.

$$\begin{pmatrix} 4 & 0 & 5 & 2 \\ 1 & 3 & -6 & 1 \\ -1 & 1 & 5 & -6 \\ -2 & 2 & -2 & 3 \end{pmatrix}$$

```
1 sortrows(A, 2)
```

Matlab

## Effect 11

Sorts the rows of **A** by their second entry in each row in ascending order.

$$\begin{pmatrix} 4 & 0 & 5 & 2 \\ -1 & 1 & 5 & -6 \\ -2 & 2 & -2 & 3 \\ 1 & 3 & -6 & 1 \end{pmatrix}$$

# MATLAB Angles

What does this M-file do? Try to predict what it will compute, then enter it into MATLAB to check.

```
1 R = rand(4,6)
2 S = sqrt(sum(R.*R))
3 for i=1:6
4     R(:,i) = R(:,i)/S(i)
5 end
```

## Answer 12

This routine converts each column vector of **R** into a unit vector.

Now your mini-project, write a Live Script which does the following:

- 1) Creates a matrix with 10 columns, each of which is a randomly generated vector in  $\mathbb{R}^3$ .
- 2) For each column vector, determines the cosine of the angle between it and the vector  $\begin{pmatrix} 1 & 2 & 3 \end{pmatrix}$ .
- 3) Ranks the vectors from 1 (the largest cosine) to 10 (the smallest cosine), and displays the rankings.

```
1 R = rand(3,10)
2 cosines = zeros(1,10);
3 ref_vec = [1;2;3];
4 ref_vec_mag = sqrt(dot(ref_vec, ref_vec));
5
6 for i=1:10
7     vec_mag = sqrt(dot(R(:,i), R(:,i))); % ith random vector magnitude
8     cosines(:,i) = dot(R(:,i), ref_vec) / (vec_mag*ref_vec_mag); %
    Cosines of the angles between each vector and the reference vector
9 end
10
    vec_cosines = transpose(sortrows(transpose(vertcat(R, cosines)), 4,
11 "descend")); % All vectors and their corresponding angles formed with the
    reference vector, sorted from greatest cosine to least
12
13 % Displaying the rankings
14 for i=1:10
15     disp(i + ") Cosine: " + vec_cosines(4,i))
16     disp(vec_cosines(1:3,i))
17 end
```